Design of an Arithmetic-Logical Floating-Point Unit in VHDL for Addition and Multiplication

student: Asaftei Alexandru-Petru
Structure of Computer Systems Project
Technical University of Cluj-Napoca

CONTENTS

1. INTRODUCTION…….....4
   1.1.Context…….....4
   1.2.Problem…….....4
   1.3.Motivation…….....4
   1.4.Objectives…….....5
   1.5.Project Plan…….....5
2. BIBLIOGRAPHIC STUDY…….....6
   2.1.Critical Analysis of Similar Existing Solution…….....6
3. ANALYSIS…….....7
   3.1.Floating-point Representation…….....7
   3.2.Addition…….....7
      3.2.1. Unpack…….....8
      3.2.2. Exponent Subtract.....8
      3.2.3. Preshift and Complement.....9
      3.2.4. Add.....9
      3.2.5. Pre-round Normalize.....9
      3.2.6. Round 9
      3.2.7. Post-round Normalize.....9
      3.2.8. Pack.....9
   3.3.Multiplication.....10
      3.3.1. Add exponent.....11
      3.3.2. Multiply.....11
      3.3.3. Pre-round Normalize.....11
   3.4.Control Unit.....11
4. DESIGN.....12
   4.1. Utilization of Theoretical Concepts.....12
   4.2. Adder.....12
   4.3. Multiplier.....12
   4.4. Control Unit.....13
   4.5. Floating Point Unit.....14
5. IMPLEMENTATION.....15
   5.1.Adder.....15
      5.1.1. Unpack.....15
      5.1.2. Exponent Subtractor.....16
      5.1.3. Align Significand.....16
      5.1.4. Normalize add.....17
      5.1.5. Round add.....18
      5.1.6. Pack.....19

5.2.Multiplier.....20
5.2.1. Unpack.....20
5.2.2. Exponent adder.....21
5.2.3. Multiplier.....22
5.2.4. Normalize.....23
5.2.5. Rounder.....24
5.2.6. Pack.....25
5.3.Floating Point Unit.....25
5.3.1. Adder.....25
5.3.2. Multiplier.....26
5.3.3. Fsm.....27
5.3.4. Im.....28
5.3.5. Reg File.....29
5.3.6. Pc.....30
6. TESTING AND VALIDATION.....32
7. CONCLUSIONS.....33
8. REFERENCES.....34

1. INTRODUCTION

1.1. Context

Floating-point arithmetic architectures are an essential component in modern digital systems, enabling precise computations for applications in fields such as scientific computing, engineering, signal processing, and graphics. Unlike fixed-point arithmetic, which has a limited dynamic range, floating-point arithmetic provides a flexible representation of real numbers, automatically adapting to very large or very small values. The IEEE 754 standard, established in 1985 and revised in 2008 and 2019, defines the binary formats for floating-point numbers, including the single-precision (32-bit) format, which is widely used due to its balance between precision and computational efficiency. In hardware design, implementing floating-point units (FPUs) allows for the creation of custom accelerators that outperform software emulations on general-purpose processors. This project focuses on designing an arithmetic-logical floating-point unit (FPU), centered on addition and multiplication operations, compliant with the IEEE 754 standard in single-precision format, as these operations form the foundation for more complex calculations and are critical in real-time systems.

1.2. Problem

The primary challenge in designing an FPU lies in the complexity of floating-point operations compared to integer arithmetic. For addition, difficulties include aligning mantissas based on differing exponents, managing sign differences (which may convert addition into subtraction), normalizing the result, and handling special cases such as Not-a-Number (NaN), infinity, zero, and denormalized numbers. Multiplication involves multiplying mantissas, adding exponents with bias correction, and similar normalization alongside special case management. These operations must adhere to the IEEE 754 standard to ensure portability and accuracy, but hardware implementations often face high latency, significant resource usage, and power consumption. The problem is compounded by the need to balance precision, performance, and area, as naive implementations can lead to inaccuracies due to rounding errors or inefficiencies in processing.

1.3. Motivation

The motivation arises from the growing demand for efficient hardware acceleration in edge computing and IoT devices, where software-based floating-point operations on microcontrollers are too slow or power-intensive. For example, in signal processing applications such as audio processing or sensor fusion, real-time floating-point computations are critical. Commercial FPUs exist in processors like ARM Cortex or Intel x86, but custom designs allow tailoring to specific needs, such as reduced precision for lower power or integration into larger systems-on-chip (SoCs). This project addresses the lack of open-source, educational implementations that are both IEEE 754-compliant and optimized, providing a foundation for students and engineers. Moreover, with the increasing accessibility of programmable devices, there is a need for verifiable projects that demonstrate best practices in addressing the inherent challenges of floating-point arithmetic.

1.4. Objectives

The main objective is to develop a synthesizable module for an arithmetic-logical floating-point unit that performs addition and multiplication according to IEEE 754 in single precision, with support for special cases and rounding to the nearest even. Secondary objectives include verifying the design through simulation, synthesizing it on a programmable device, and evaluating performance metrics such as latency, throughput, and resource usage. To address the identified problem, the project adopts a modular architecture:

- **Unpacking**: Extracting the sign, biased exponent (8 bits), and mantissa (23 bits with an implicit leading 1).
- **Special Case Handling**: Early detection of NaN, infinity, or zero to bypass normal computation and produce correct outputs (e.g., NaN propagation, infinity rules).
- **For Addition**: Comparing exponents to align the smaller mantissa by right-shifting, performing signed addition/subtraction, normalizing the result by left-shifting to restore the leading 1 with exponent adjustment, and applying rounding with guard, round, and sticky bits.
- **For Multiplication**: Determining the result sign via XOR, adding exponents with bias subtraction (127), multiplying mantissas, normalizing the product (shifting if MSB=1 and incrementing the exponent), and applying similar rounding.
- **Packing**: Reassembling into a 32-bit format, with overflow/underflow checks to saturate to infinity or zero. This step-by-step approach ensures compliance and efficiency, with parallel processing introduced in later stages to reduce critical path delay.

1.5. Project Plan

Meeting 1: Finalize Introduction and Bibliographic Study. Targets: Consolidate the context, problem statement, motivation, objectives, and project plan in the Introduction, and complete the critical analysis of existing solutions in the Bibliographic Study.

Meeting 2: Complete Analysis and Design chapters. Targets: Develop detailed theoretical descriptions of floating-point representation, addition, multiplication, and pipeline processing in Analysis, and define the system architecture and component interactions in Design.

Meeting 3: Complete Implementation and Testing and Validation chapters. Targets: Detail the realization of the design in Implementation, and outline the evaluation process and validation methods in Testing and Validation.

Meeting 4: Finalize Conclusions and References. Targets: Summarize findings, propose future enhancements in Conclusions, and compile all sources in References using the agreed bibliography style.

2. BIBLIOGRAPHIC STUDY

2.1. Critical Analysis of Similar Existing Solution

Existing solutions for floating-point arithmetic units primarily focus on IEEE 754 compliance, targeting programmable devices for their reconfigurability and parallel processing capabilities. Theoretical concepts central to development include the IEEE 754 binary format (sign, exponent with bias, normalized mantissa), rounding modes (e.g., round-to-nearest-even to minimize bias), exception handling (overflow, underflow, inexact), and optimization techniques such as parallel processing and fast adders to reduce latency. Others address the problem by breaking operations into stages: unpacking, alignment/multiplication, normalization, rounding, and packing, often leveraging specialized blocks for mantissa operations. Advantages of these designs include high speed (e.g., through autonomous or parallel architectures) and synthesizability, but drawbacks encompass significant resource usage, complexity in handling special cases that increases area, and potential inaccuracies if denormals or rounding are simplified. Compared to fixed-point arithmetic, floating-point offers a better dynamic range but at the cost of slower operations and higher power consumption; surveys highlight that fixed-point is preferred in signal processing unless wide ranges are required, where floating-point's automatic scaling excels but requires careful precision management to avoid rounding errors.

In [0] the authors propose a high-performance IEEE 754 floating-point multiplier using optimized architectures for speed; advantages include energy efficiency, but drawbacks involve complexity in timing verification. A similar solution is presented in [1], where a fast adder architecture based on optimized designs is described; it excels in performance but suffers from higher resource demands without additional optimizations. In [2] an implementation of a single-precision adder compliant with IEEE 754 is presented, emphasizing optimal area and high performance; it offers good accuracy but notes increased delays from rounding logic. The work in [3] details a 32-bit floating-point multiplier and adder for FFT processing, following the IEEE 754 standard; advantages lie in specific applications, but portability to other platforms is limited. In [4] a floating-point multiplier is proposed, addressing the challenges of complex operations; it benefits from efficiency in normal cases, but critiques include the omission of denormals for simplicity, leading to inaccuracies in subnormal ranges. A survey in [5] analyzes floating-point unit designs, noting that parallel approaches reduce latency but increase resource usage; it recommends optimized multipliers for multiplication-intensive applications. Similarly, [6] reviews floating-point adders, highlighting the advantages of fast adders for alignment but drawbacks in power consumption compared to simpler variants. In [7] floating-point processors are discussed, with benefits in vector operations but challenges in hardware reuse for different formats. The analysis in [8] explains the difficulties of floating-point design, such as special case handling that inflates complexity, making it more challenging than fixed-point. Finally, [9] compares fixed vs. floating-point arithmetic, noting the dynamic range advantage of the latter but the drawback of higher latency, suggesting hybrid approaches for optimization. These works inform our project by adopting modular and parallel structures, addressing gaps such as full denormal support for improved accuracy.

3. ANALYSIS

3.1. Floating-Point Representation

Floating-point numbers are represented in a format that supports a wide range of values with adjustable precision, critical for scientific and engineering applications. In the single-precision format, a number is represented as a sign bit (1 bit: 0 for positive, 1 for negative), an exponent (8 bits: biased by 127, where the actual exponent is the stored value minus 127), and a mantissa (23 bits: normalized with an implicit leading 1, providing effective 24 bits). Special cases are handled first to avoid invalid computations.

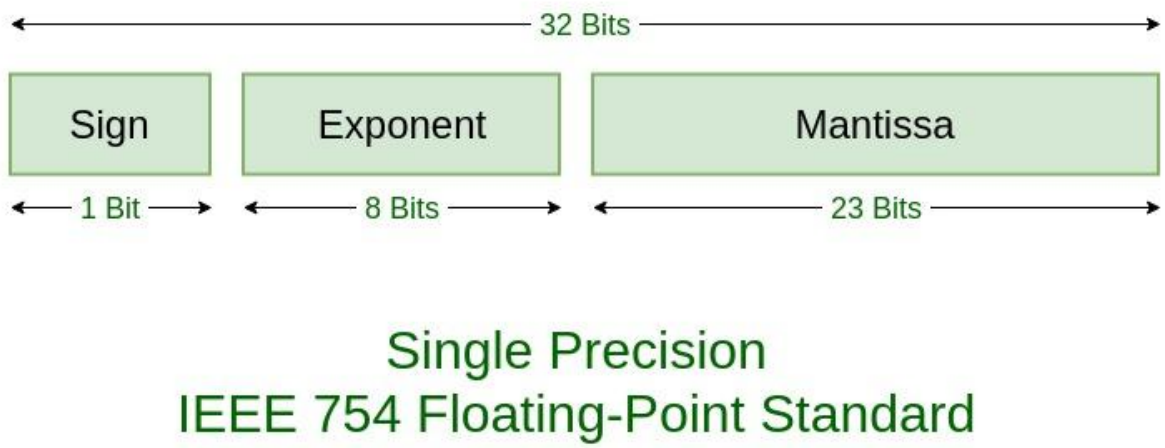


Figure 1. Single-precision floating-point representation. As depicted in Figure 1, the layout consists of the sign bit, biased exponent, and mantissa. [10]

3.2. Addition

Addition in floating-point arithmetic is the most complex operation, involving multiple steps where exceptions can occur and checks are required to ensure accuracy. It is performed in a natural way by aligning operands to the same exponent, computing their sum, normalizing, and rounding the result. The process is divided into the following steps: unpack the operands into signs, exponents, and mantissas with implicit 1; handle special cases like NaN or infinity; align mantissas by shifting the smaller one right by the exponent difference and setting the result exponent to the maximum; add or subtract mantissas based on signs using two's complement for subtraction; normalize by shifting the result mantissa left or right to achieve a leading 1 and adjust the exponent; round to nearest using guard bits and pack into 32 bits; check for overflow/underflow if the exponent exceeds 254 or falls below 1.

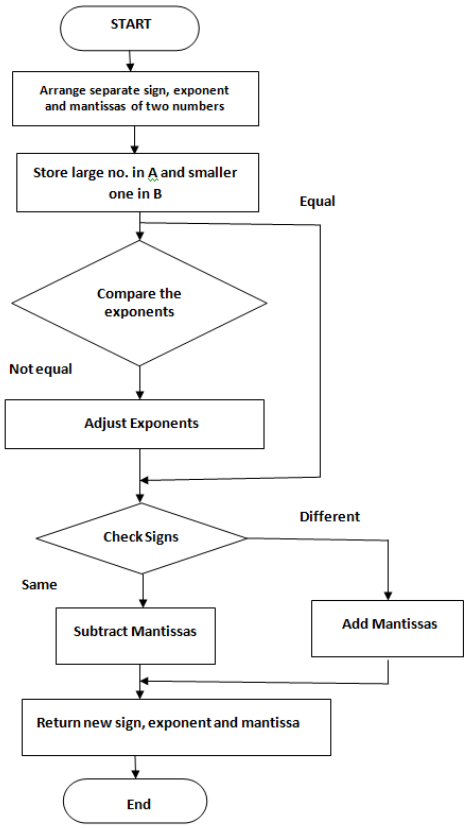


Figure 2. Flowchart of IEEE 754 floating-point addition algorithm. As shown in Figure 2, the flowchart outlines the sequence of unpacking, alignment, addition/subtraction, normalization, rounding, and packing. [11]

3.2.1. Unpack

This is one of the simplest stages, where the components of the operands are separated, and the hidden leading 1 is reinstated in the significand. The inputs are the operands, and the outputs are their defining features: sign (1-bit), exponent (8-bits), and significand (23-bits). These results are forwarded to the next stage.

3.2.2. Exponent Subtract

The exponents are subtracted to determine the difference. Based on the sign of this difference, the tentative exponent is chosen, and the operand requiring alignment is identified. A flag indicates if the alignment will destroy the operand by shifting all bits out.

3.2.3. Preshift and Complement

One operand supports preshift, and the other complementation. The operand with the smaller exponent is shifted right by the difference amount. The other operand is negated if signs differ. It is possible that operands are swapped if the one needing alignment lacks preshift support. Three bits for normalization and rounding are computed: guard (most-significant shifted-out bit), round (second most-significant shifted-out bit), and sticky (logical OR of remaining lost bits). This produces 32-bit numbers forwarded to the addition stage.

3.2.4. Add

A fast addition mechanism handles the aligned and possibly complemented mantissas in two's complement. The carry-in depends on the operand signs. The initial result is complemented if negative and sent for normalization to obtain a standard-recognized significand.

3.2.5. Pre-round Normalize

Since operands are normalized in [1, 2), after preshift they are in (-2, 2), with the sum in (-4, 4). After addition and negation, the initial sum is in [0, 4). If greater than or equal to 2, it is shifted right by one, incrementing the tentative exponent. If less than 1 but not zero, it is shifted left until a 1 is before the radix point002C decrementing the exponent by the shift amount (number of leading zeros). Shifting modifies the round and sticky bits, with the guard bit protecting against precision loss during left shifts.

3.2.6. Round

Based on the rounding mode, round and sticky bit values, rounding is performed by adding a unit in the last place if needed. Since this may modify the significand, another normalization step follows.

3.2.7. Post-round Normalize

After rounding, the result may become greater than or equal to 2, requiring detection and right shifting by one bit, modifying the tentative exponent accordingly.

3.2.8. Pack

The final step assembles the result components. A check determines if the result is zero, which has a special representation. Final overflow and underflow bits are generated based on flags from previous exponent-modifying steps.

3.3 Multiplication

Multiplication is simpler than addition, as no preshift alignment is needed; exponents are added, and significands multiplied. The process is divided into unpacking (identical to addition), adding exponents, multiplying, pre-round normalization, rounding and normalization (identical to addition), and packing (identical to addition).

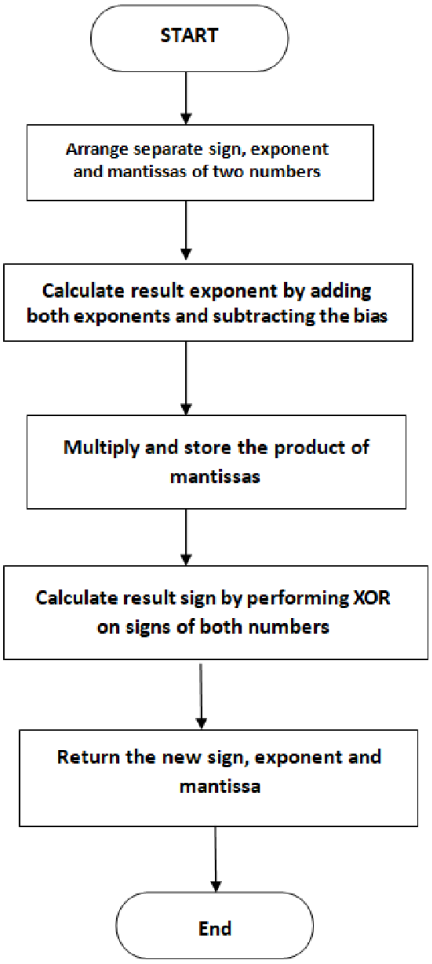


Figure 3. Flowchart of IEEE 754 floating-point multiplication algorithm. As illustrated in Figure 3, the flowchart details the steps of unpacking, exponent addition, multiplication, pre-round normalization, rounding, and packing. [12]

1. Unpack (indentical to addition)
2. Add Exponents
3. Multiply
4. Pre-round Normalize
5. Round and Normalize (indentical to addition)
6. Pack (indentical to addition)

3.3.1. Add Exponents

Exponents are added, and the bias subtracted. Overflow or underflow may occur, outputting required flags.

3.3.2. Multiply

The 24-bit significands are multiplied, producing a 48-bit product forwarded to normalization.

3.3.3. Pre-round Normalize

Significands in [1, 2) mean the product is in [1, 4). Thus, at most one right shift and exponent increment is needed, outputting an overflow flag.

3.4 Control Unit

The control unit is a finite-state machine that waits for a 48-bit instruction reception, then decodes it. Operands are fetched, and the proper future state is selected based on the operation. An execute state follows, ending with a write or transmit state.

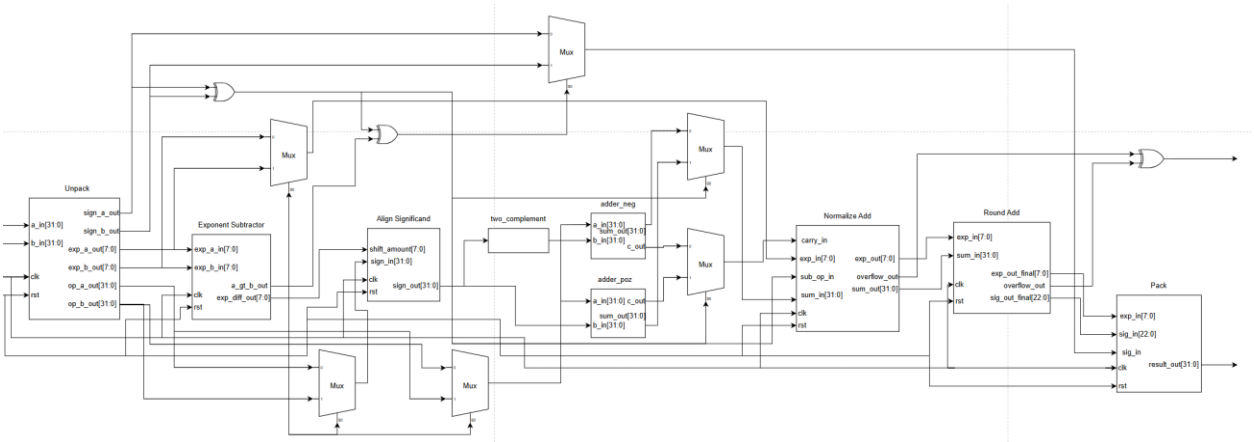
4. DESIGN

4.1 Utilization of Theoretical Concepts

The design applies the floating-point representation by processing sign, exponent, and mantissa independently across stages. The addition and multiplication algorithms define modular components, with unpacking, exponent handling, computation, normalization, rounding, and packing directly mapped to ensure compliance with precision and exception rules. Pipeline processing enables concurrent execution of these stages, while the control unit's finite-state machine correlates with instruction types and execution cycles to manage data flow and operation sequencing.

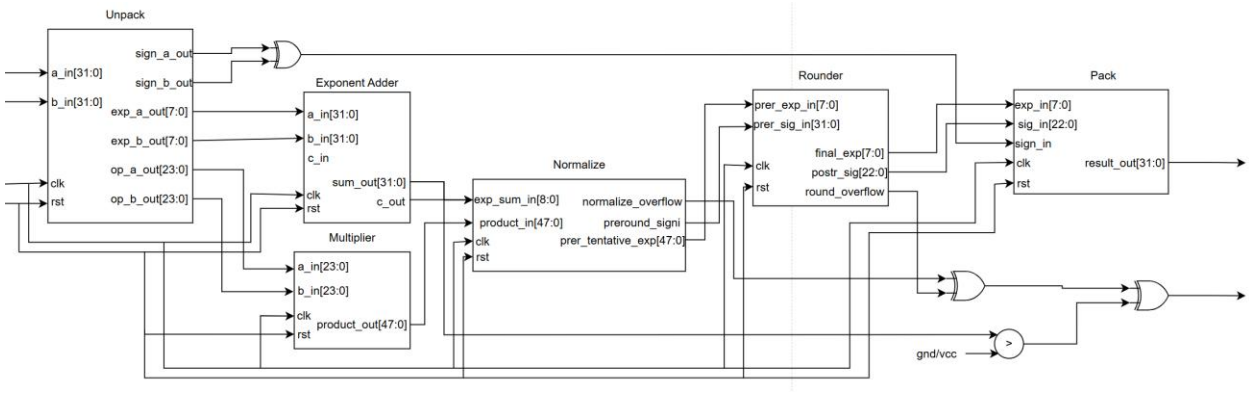
4.2. Adder

The adder includes modules for each addition stage: unpacking operands, exponent subtraction to determine alignment, preshift and complement for mantissa preparation, addition of aligned mantissas, pre-round normalization to adjust the sum, rounding based on guard bits, post-round normalization if needed, and packing the final result.



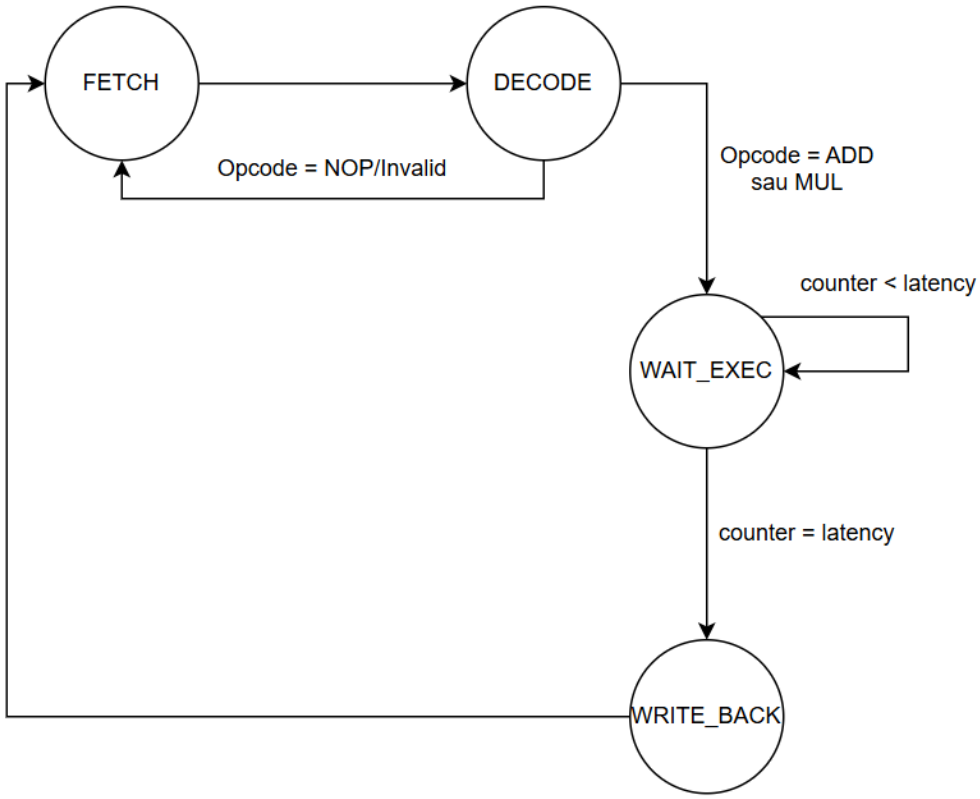
4.3. Multiplier

The multiplier consists of modules for unpacking operands, adding exponents with bias correction, multiplying mantissas, pre-round normalization of the product, rounding with potential normalization adjustment, and packing the result.



4.4. Control Unit

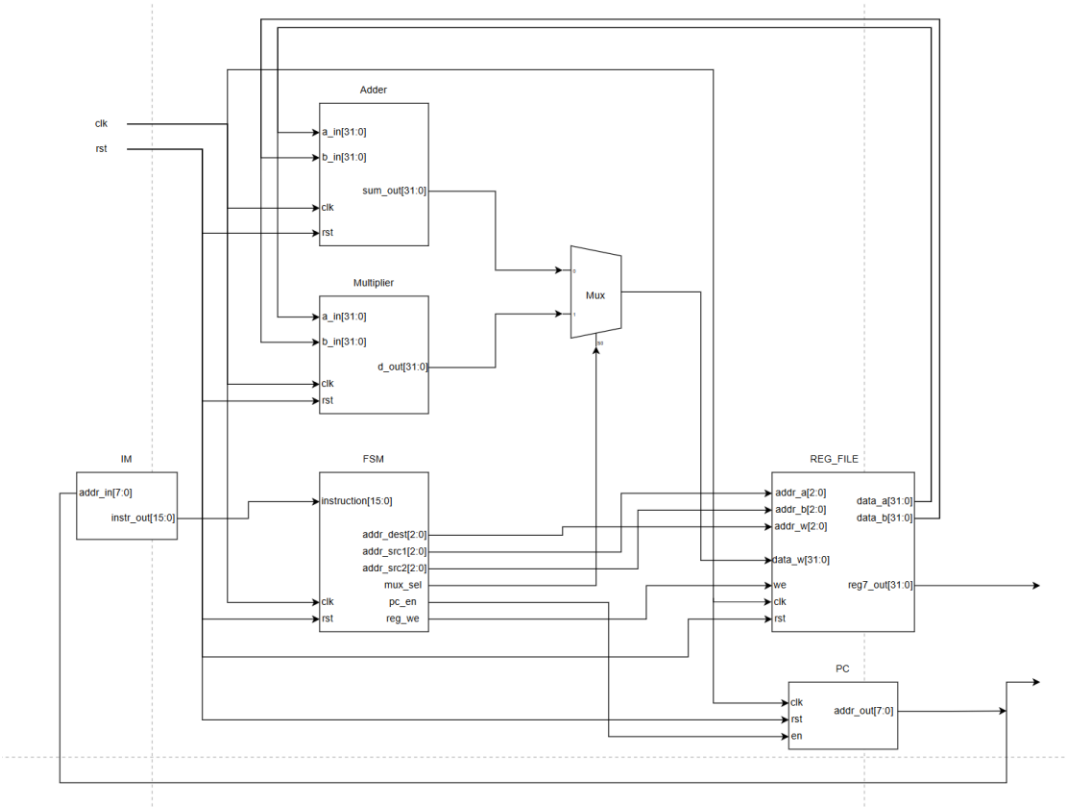
The control unit is a finite-state machine with states for receiving instructions, decoding them, fetching operands, executing operations like add or multiply, writing results, and transmitting values if required.



STATE	FUNCTION
FETCH	Instruction Fetch. Waits for the instruction to be read from the Instruction Memory (IM) at the current address pointed to by the Program Counter (PC).
DECODE	Instruction Decode. The Opcode is analyzed (00 or 10). The control signals (mux_sel, alu_sub) are configured, and the latency counter is initialized (8 cycles for ADD, 6 cycles for MUL).
WAIT_EXEC	Pipeline Execution Wait. The FSM remains in this state, incrementing an internal counter until the data propagates through all pipeline stages of the selected arithmetic unit.
WRITE_BACK	Write-Back. The reg_we signal is asserted to save the final result into the Register File, and the pc_en signal is asserted to increment the Program Counter to the next instruction.

4.5. Floating Point Unit

The FPU Top Level integrates a control unit, instruction memory, and register file with pipelined arithmetic modules to execute floating-point operations. It orchestrates the fetch-decode-execute cycle, managing data flow between storage and the parallel adder and multiplier units.

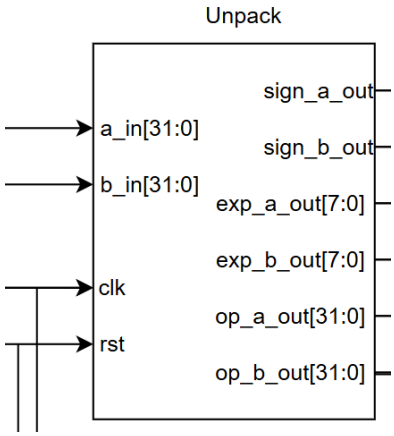


5. IMPLEMENTATION

5.1. Adder

5.1.1. Unpack

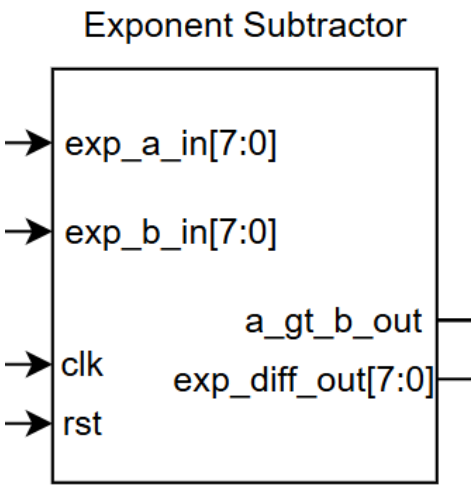
This module "unpacks" two 32-bit floating-point numbers to prepare them for an addition/subtraction pipeline. It extracts the sign and exponent, then builds a 32-bit internal operand (op_a_out, op_b_out) by prepending the hidden bit ('1' or '0') to the 23-bit mantissa and appending 8 zeros at the end. These appended zeros serve as the initial Guard, Round, and Sticky bits that are required for the alignment (shifting) and rounding stages later in the adder.



PORTS	DIRECTION	DESCRIPTION
a_in	in	Operand a
b_in	in	Operand b
sign_a_out	out	Sign of a
sign_b_out	out	Sign of b
exp_a_out	out	Exponent of a
exp_b_out	out	Exponent of b
op_a_out	out	Significand of a
op_b_out	out	Significand of b
clk	in	Clock
rst	in	Reset

5.1.2. Exponent Subtractor

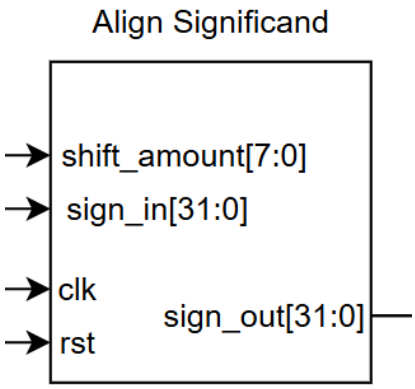
This module compares two 8-bit exponents (exp_a_in and exp_b_in) to determine which is larger and by how much. It outputs the absolute difference between them (exp_diff_out) and a flag (a_gt_b_out) that is '1' if exp_a_in is greater than exp_b_in. This logic is the core of the significand alignment stage in a floating-point adder, as it provides the shift amount and determines which significand needs to be shifted.



PORTS	DIRECTION	DESCRIPTION
exp_a_in	in	Exponent of operand a
exp_b_in	in	Exponent of operand b
exp_diff_out	out	Absolute difference of exponents (|a - b|)
a_gt_b_out	Out	Flag: '1' if exp_a > exp_b, else '0'
clk	in	Clock
rst	in	Reset

5.1.3. Align Significand

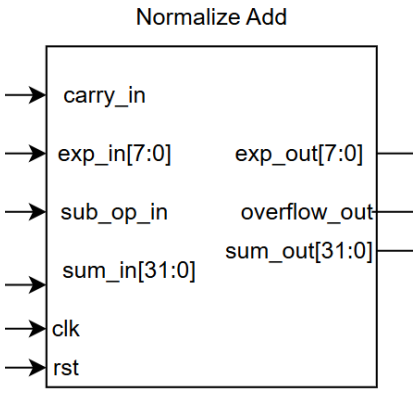
This module performs the significand alignment step, which is essential before adding or subtracting floating-point numbers. It takes the 32-bit internal significand (sig_in) of the number with the *smaller* exponent and shifts it to the right by the amount specified in shift_amount (which is the exponent difference). This operation ensures both significands are aligned to the same larger exponent, allowing them to be correctly added or subtracted.



PORTS	DIRECTION	DESCRIPTION
sig_in	in	The input significand (mantissa) to be shifted
shift_amount	in	The number of positions to shift the significand to the right
sig_out	out	The resulting significand after the right shift
clk	in	Clock
rst	in	Reset

5.1.4. Normalize Add

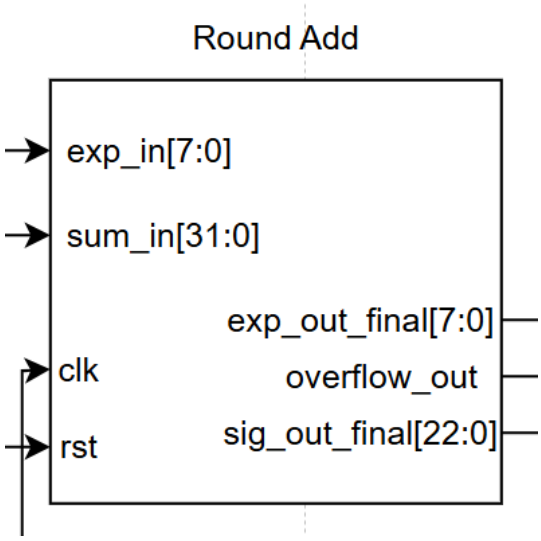
This module is the normalization stage for the floating-point adder, correcting the result to the standard 1.xxxx... format after the significands are added or subtracted. If an addition causes an overflow (detected by carry_in), it shifts the sum right by one bit and increments the exponent. Conversely, if a subtraction causes underflow (losing the leading '1'), it shifts the sum left until the leading '1' is restored and decrements the exponent accordingly.



PORTS	DIRECTION	DESCRIPTION
sum_in	in	The unnormalized significand sum from the previous stage
carry_in	in	The carry-out bit from the significand adder, used to detect overflow
exp_in	in	The exponent corresponding to the input sum
sub_op_in	in	Flag indicating the operation ('0' = Add, '1' = Subtract), used to correctly
sum_out	out	The final normalized significand
exp_out	out	The final adjusted (normalized) exponent
overflow_out	out	Flag set to '1' if an overflow (requiring a right shift) occurred during normalization
clk	in	Clock
rst	in	Reset

5.1.5. Round Add

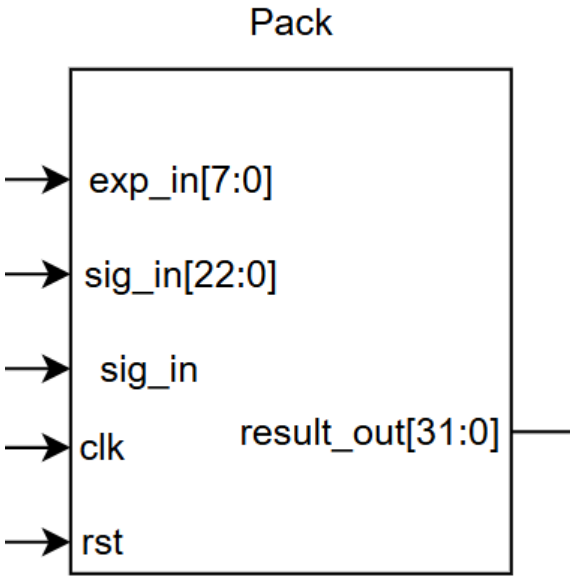
This module performs the final **rounding** step for the floating-point **adder**. It takes the 32-bit normalized sum (which now contains information in the 8 extra bits from alignment and normalization) and uses the **Guard**, **Round**, and **Sticky** bits to apply the IEEE 754 "round-to-nearest-even" logic. It truncates the significand to the final 23 bits and adjusts the exponent if the rounding operation causes an overflow (e.g., `1.11...1` rounds up to `10.0...0`), while also handling special exponent cases for Infinity and Zero.



PORTS	DIRECTION	DESCRIPTION
sum_in	in	The normalized significand, including guard, round, and sticky bits from the previous stage
exp_in	in	The exponent corresponding to the normalized sum
sig_out_final	out	The final, rounded 23-bit significand (mantissa) ready for output
exp_out_final	out	The final exponent, adjusted if rounding caused an overflow
overflow_out	out	Flag set to '1' if the rounding operation caused the significand to overflow
clk	in	Clock
rst	in	Reset

5.1.6. Pack

This module performs the final "packing" stage of the floating-point pipeline. It assembles the 32-bit single-precision floating-point result by concatenating the final sign bit, the 8-bit exponent, and the 23-bit significand, matching the IEEE 754 standard format.

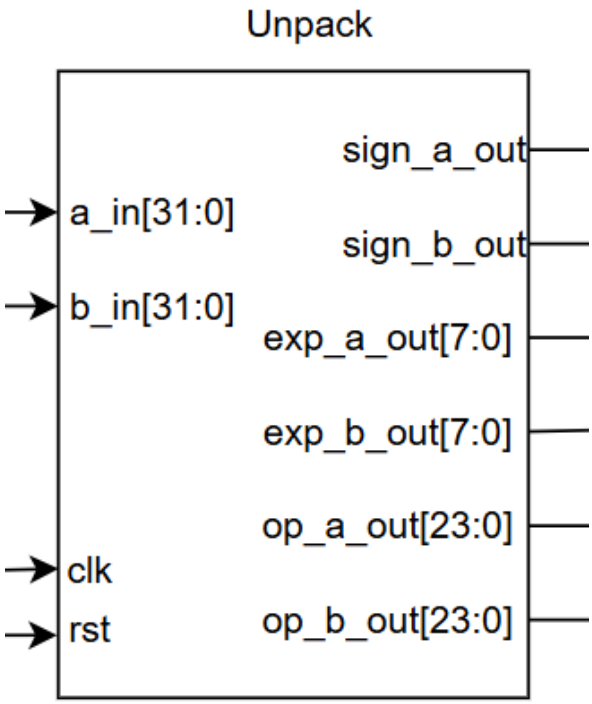


PORTS	DIRECTION	DESCRIPTION
sign_in	in	The final sign bit (0 for positive, 1 for negative)
exp_in	in	The final 8-bit exponent
sig_in	in	The final 23-bit significand (mantissa)
result_out	out	The combined 32-bit single-precision floating-point result
clk	in	Clock
rst	in	Reset

5.2. Multiplier

5.2.1. Unpack

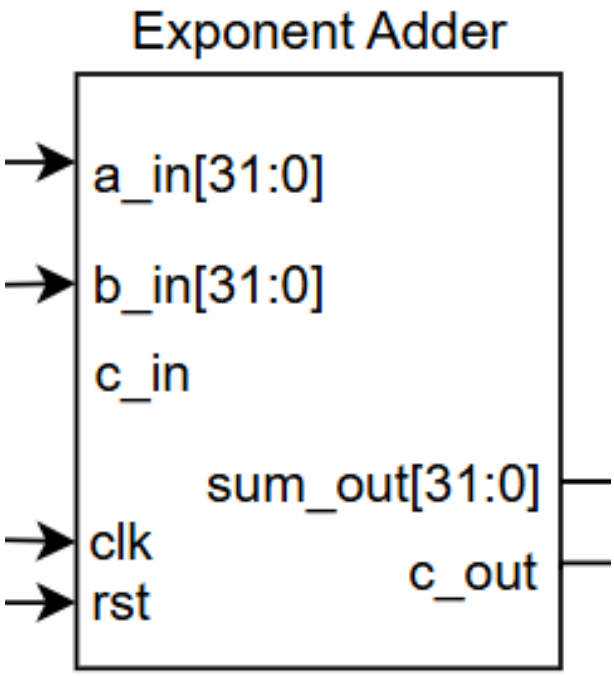
This module "unpacks" two 32-bit floating-point input numbers into their core components: sign, exponent, and the 24-bit significand, adding the implied hidden bit to the significand to prepare it for calculations.



PORTS	DIRECTION	DESCRIPTION
a_in	in	Input operand a
b_in	in	Input operand b
sign_a_out	out	The sign bit extracted from operand a
sign_b_out	out	The sign bit extracted from operand b
exponent_a_out	out	The 8-bit exponent extracted from operand a
exponent_b_out	out	The 8-bit exponent extracted from operand b
significand_a_out	out	The 24-bit significand (mantissa) of a, including the hidden bit
significand_b_out	out	The 24-bit significand (mantissa) of b, including the hidden bit
clk	in	Clock
rst	in	Reset

5.2.2. Exponent adder

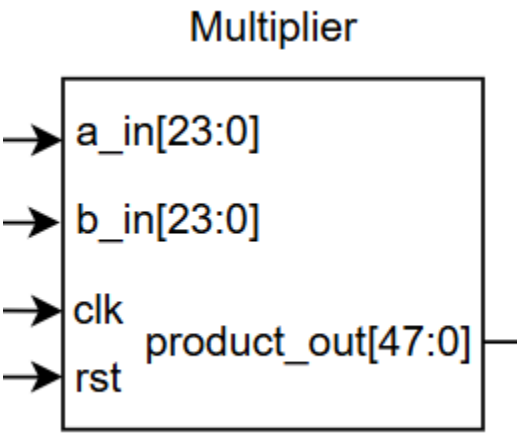
It simply uses the + operator from the numeric_std library to add two 8-bit operands (a_in, b_in) and a carry-in bit (c_in). It generates the 8-bit sum (sum_out) and the carry-out bit (c_out).



PORTS	DIRECTION	DESCRIPTION
a_in	in	Input operand a
b_in	in	Input operand b
c_in	in	Carry-in bit
sum_out	out	The 8-bit sum result
c_out	out	The carry-out bit
clk	in	Clock
rst	in	Reset

5.2.3. Multiplier

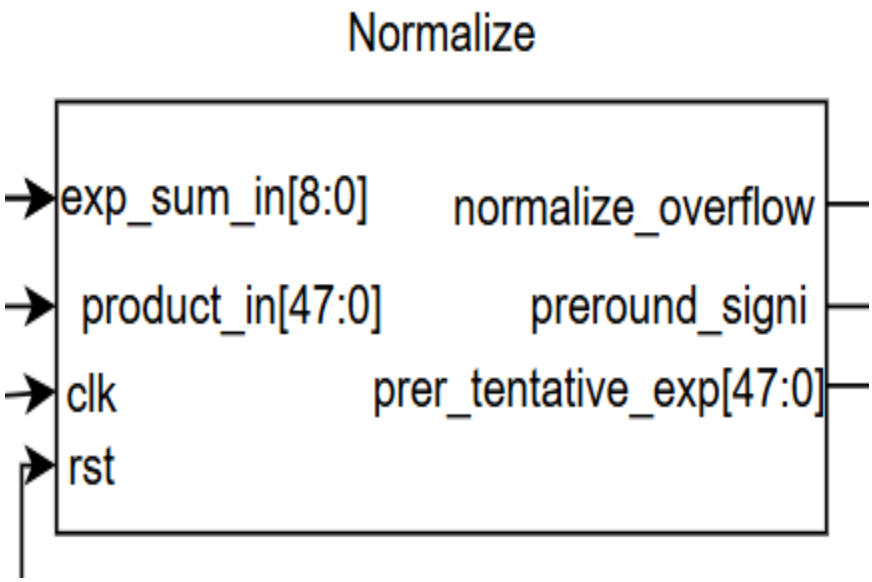
It uses the * operator from the numeric_std library to multiply two 24-bit unsigned operands (a_in, b_in) and generate the 48-bit product (product_out).



PORTS	DIRECTION	DESCRIPTION
a_in	in	Input operand a (24-bit)
b_in	in	Input operand b (24-bit)
product_out	out	The 48-bit multiplication result (product)
clk	in	Clock
rst	in	Reset

5.2.4. Normalize

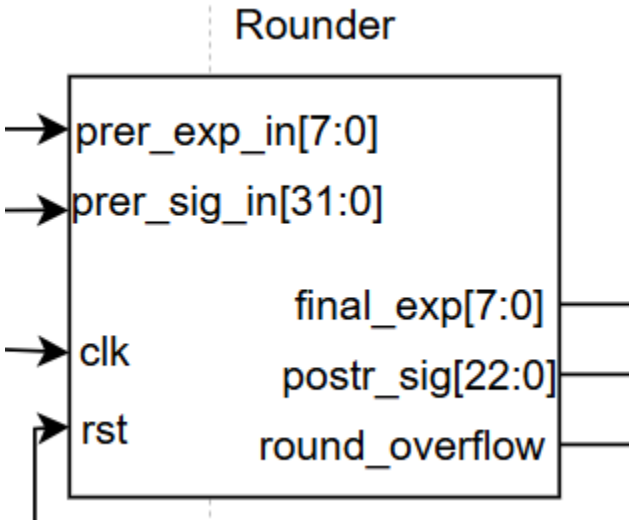
This module is the normalization stage for the multiplier. After multiplying the 24-bit significands (which results in a 48-bit product), this block's job is to adjust the result back into the standard 1.xxxx... format.



PORTS	DIRECTION	DESCRIPTION
product_in	in	The 48-bit result from the 24-bit x 24-bit significand multiplier
exp_sum_in	in	The raw sum of the two input exponents. It is 9 bits to accommodate a potential carry
preround_tentative_exponent	out	The new, adjusted 8-bit exponent, *before* the rounding stage (which could still increment it)
preround_significand	out	The 32-bit normalized significand, including the 24-bit mantissa and 8 bits for rounding
normalize_overflow	out	A flag that is set to '1' if the adjusted exponent is too large (greater than 255)
clk	in	Clock
rst	in	Reset

5.2.5. Rounder

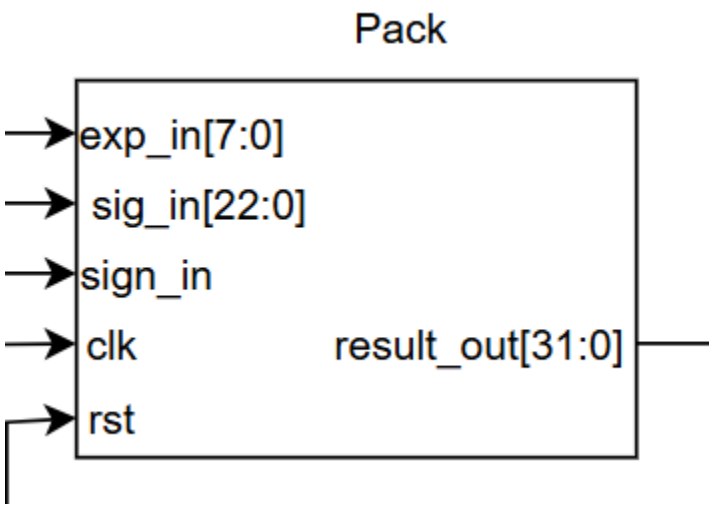
This module performs the final rounding step for the multiplication result. It takes the 32-bit significand (which includes 8 extra precision bits) from the normalization stage and uses the Guard, Round, and Sticky bits to perform "round-to-nearest-even," the default for IEEE 754. It truncates the significand to the final 23 bits and adjusts the exponent if the rounding causes an overflow (e.g., 1.11...1 rounds up to 10.0...0). It also handles special cases for exponent overflow (to Infinity) or underflow (to Zero).



PORTS	DIRECTION	DESCRIPTION
preround_exp_in	in	The 8-bit normalized exponent from the previous stage
preround_sig_in	in	The 32-bit normalized significand, which includes the 24-bit mantissa plus 8 guard/round/sticky bits
postround_significand	out	The final, rounded 23-bit significand (mantissa) ready for packing
final_exponent	out	The final 8-bit exponent, adjusted if rounding caused an overflow.
round_overflow	out	A flag set to '1' if the rounding operation caused the significand to overflow (requiring an exponent increment).
clk	in	Clock
rst	in	Reset

5.2.6. Pack

This module performs the final "packing" stage of the floating-point pipeline. It assembles the 32-bit single-precision floating-point result by concatenating the final sign bit, the 8-bit exponent, and the 23-bit significand, matching the IEEE 754 standard format.

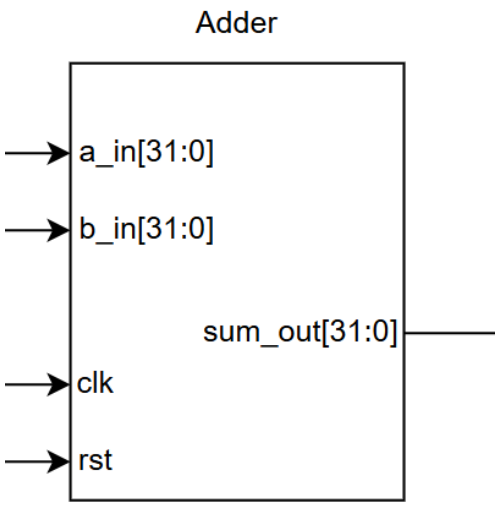


PORTS	DIRECTION	DESCRIPTION
sign_in	in	The final sign bit (0 for positive, 1 for negative)
exp_in	in	The final 8-bit adjusted exponent
sig_in	in	The final 23-bit rounded significand (mantissa)
result_out	out	The packed 32-bit single-precision floating-point result
clk	in	Clock
rst	in	Reset

5.3. Floating Point Unit

5.3.1. Adder

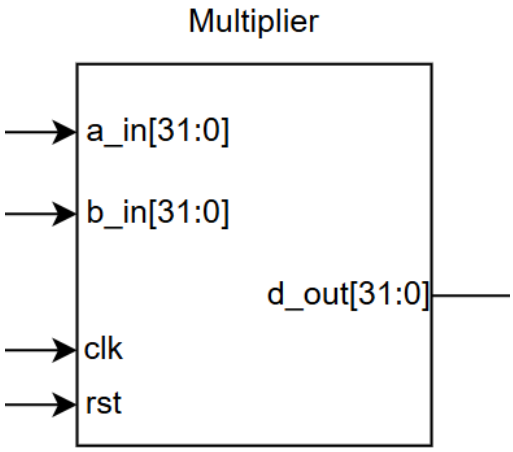
This entity represents the top-level wrapper for the entire Floating Point Adder unit. It encapsulates all the previous stages we discussed (Unpack, Align, Normalize, Round, Pack). The presence of the clk (clock) and rst (reset) ports signifies that this is a synchronous circuit, meaning the addition process is likely pipelined across multiple clock cycles to maximize processing speed and throughput.



PORTS	DIRECTION	DESCRIPTION
a_in[31:0]	in	The first 32-bit floating-point operand
b_in[31:0]	in	The second 32-bit floating-point operand
sum_out	out	The final 32-bit floating-point result of the addition
clk	in	Clock
rst	in	Reset

5.3.2. Multiplier

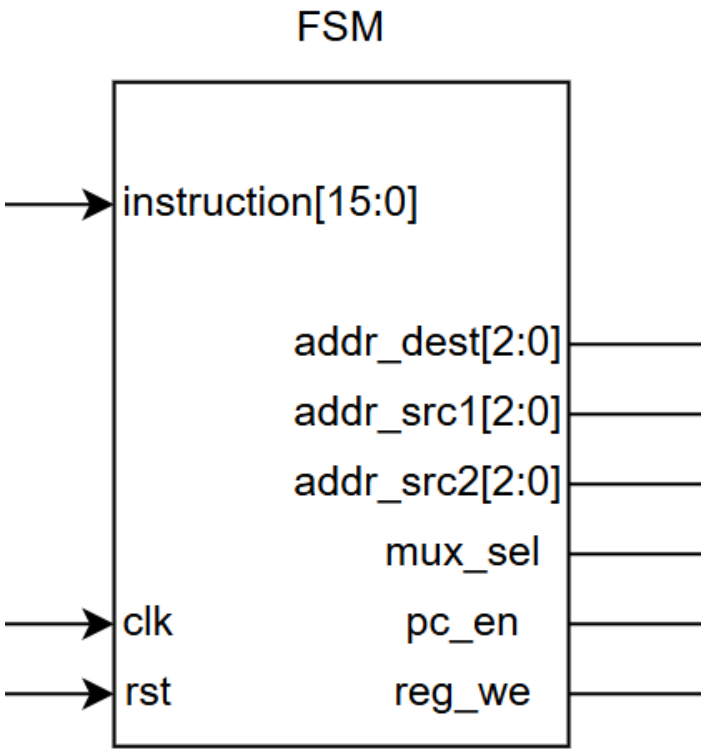
This entity is the top-level wrapper for the Floating Point Multiplier unit. It connects all the multiplication stages (Unpack, Wallace Tree Multiplier, Normalize, Round, Pack). It takes two 32-bit inputs, processes them through the pipeline stages (likely over several clock cycles), and outputs the calculated product.



PORTS	DIRECTION	DESCRIPTION
a_in[31:0]	in	The first 32-bit floating-point operand
b_in[31:0]	in	The second 32-bit floating-point operand
d_out	out	The final 32-bit floating-point result of the multiplication
clk	in	Clock
rst	in	Reset

5.3.3. Fsm

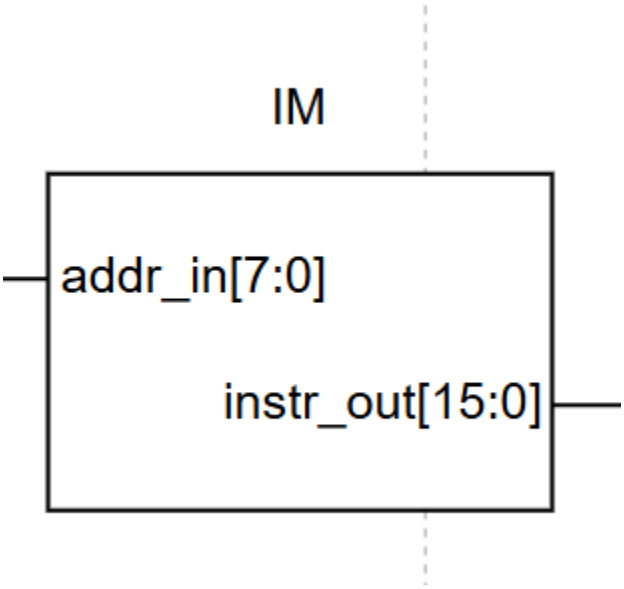
This entity acts as the Control Unit (or Decoder) of the processor. It takes the raw 16-bit binary instruction and "decodes" it to extract the register addresses (addr_src, addr_dest). At the same time, based on the Opcode (the part of the instruction that says "Add" or "Multiply"), it generates the necessary control signals (mux_sel, reg_we, pc_en) to coordinate the rest of the hardware (the Datapath) to perform the correct operation.



PORTS	DIRECTION	DESCRIPTION
instruction[31:0]	in	The 16-bit machine code instruction fetched from memory to be executed
addr_dest[2:0]	in	The 3-bit address of the Destination register (extracted from the instruction)
addr_src1[2:0]	out	The 3-bit address of the first Source register
addr_src2[2:0]	out	The 3-bit address of the second Source register
mux_sel	out	A control signal used to select data paths
pc_en	out	Program Counter Enable. A control signal that allows the processor to move to the next instruction
reg_we	out	Register Write Enable. A control signal that allows the calculated result to be written back into the register file
clk	in	Clock
rst	in	Reset

5.3.4. Im

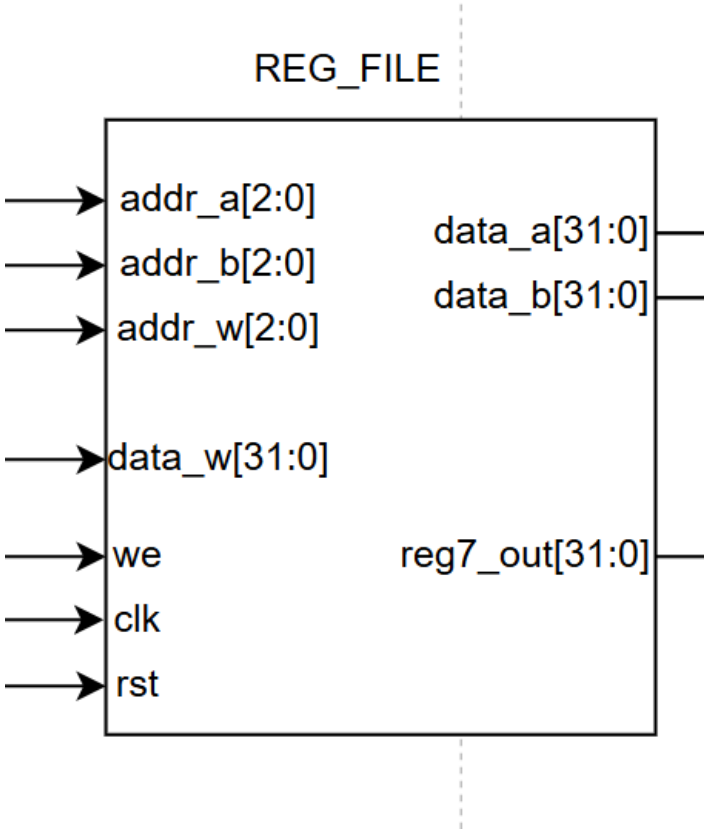
This entity represents the Instruction Memory (IM) of your processor. It acts as a Read-Only Memory (ROM) that stores the program you want the processor to run. It functions as a simple look-up table: you provide an address (addr_in), and it instantly provides the instruction stored at that location (instr_out). This is the first step of the "Fetch-Decode-Execute" cycle.



PORTS	DIRECTION	DESCRIPTION
instr_out[15:0]	out	The 16-bit instruction output. This is the actual binary machine code fetched from the memory at the specified address, which will be sent to the FSM (Control Unit) for decoding
addr_in[7:0]	in	The 8-bit address input. This signal usually comes from the Program Counter (PC) and points to the specific line of code that needs to be executed next

5.3.5. Reg file

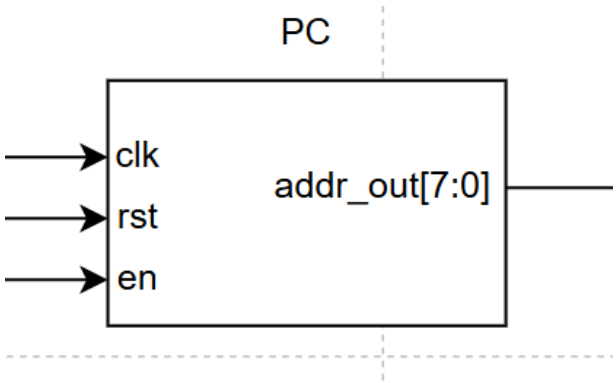
This entity is the Register File of your processor. It acts as the ultra-fast, temporary internal storage for the CPU. It is designed with dual-port read capability, meaning it can output two different values (data_a and data_b) simultaneously. This allows the processor to fetch both operands for an arithmetic operation (like A + B) in a single clock cycle. It also has a single write port to store the result back into a register when the calculation is finished.



PORTS	DIRECTION	DESCRIPTION
addr_a[2:0]	in	The 3-bit address selecting the register to be read onto port A (Source 1)
addr_b[2:0]	in	The 3-bit address selecting the register to be read onto port B (Source 2)
addr_w[2:0]	in	The 3-bit address selecting the destination register where data will be written
data_w[31:0]	in	The 32-bit data input to be written into the selected destination register
we	in	Write Enable. The control signal that permits writing to the register file. If '1', data_w is written to addr_w on the clock edge
data_a[31:0]	out	The 32-bit data output corresponding to the address addr_a
data_b[31:0]	out	The 32-bit data output corresponding to the address addr_b
reg7_out[31:0]	out	A direct output of Register 7
clk	in	Clock
rst	in	Reset

5.3.6. Pc

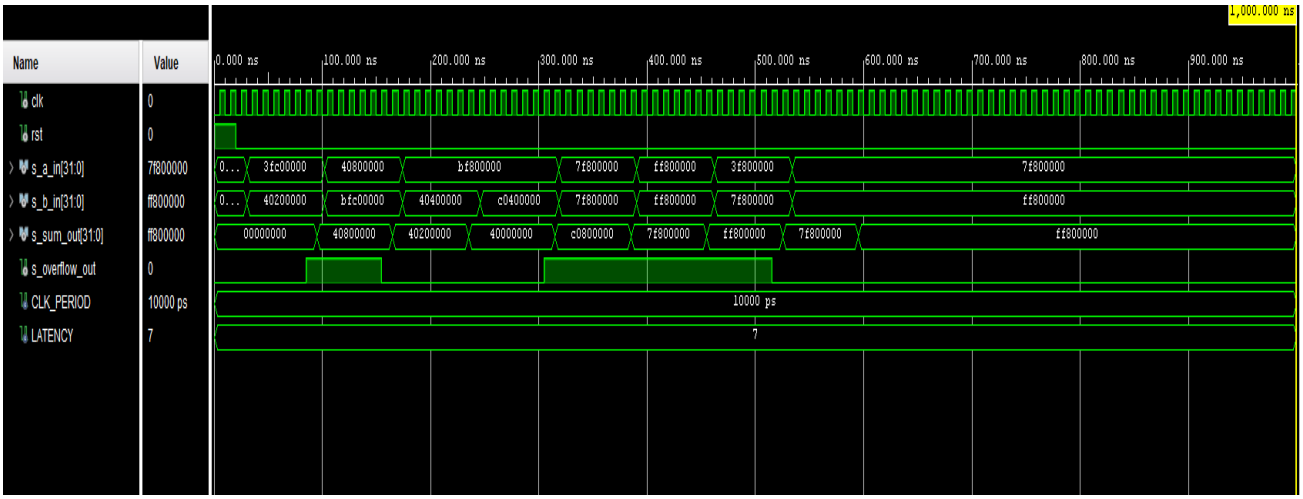
This entity is the Program Counter (PC). It acts as the "pointer" for the processor, keeping track of exactly which line of code is currently being executed. Under normal operation, every time the clock ticks (and en is active), the PC increases its value by 1. This new address is sent to the Instruction Memory to fetch the next instruction. This component is the heart of the "Fetch" stage in the processor's cycle.

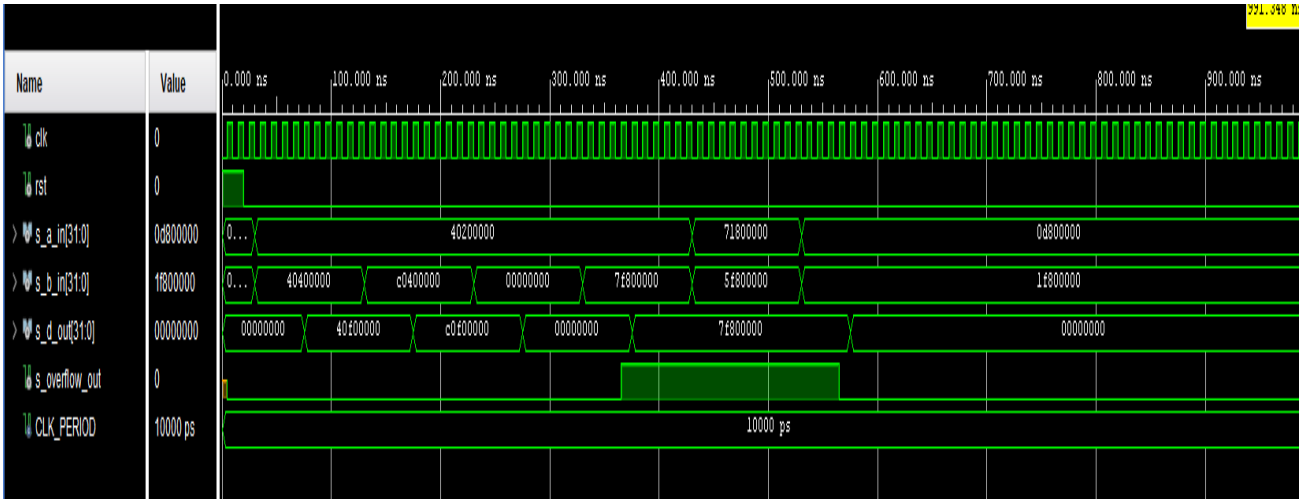


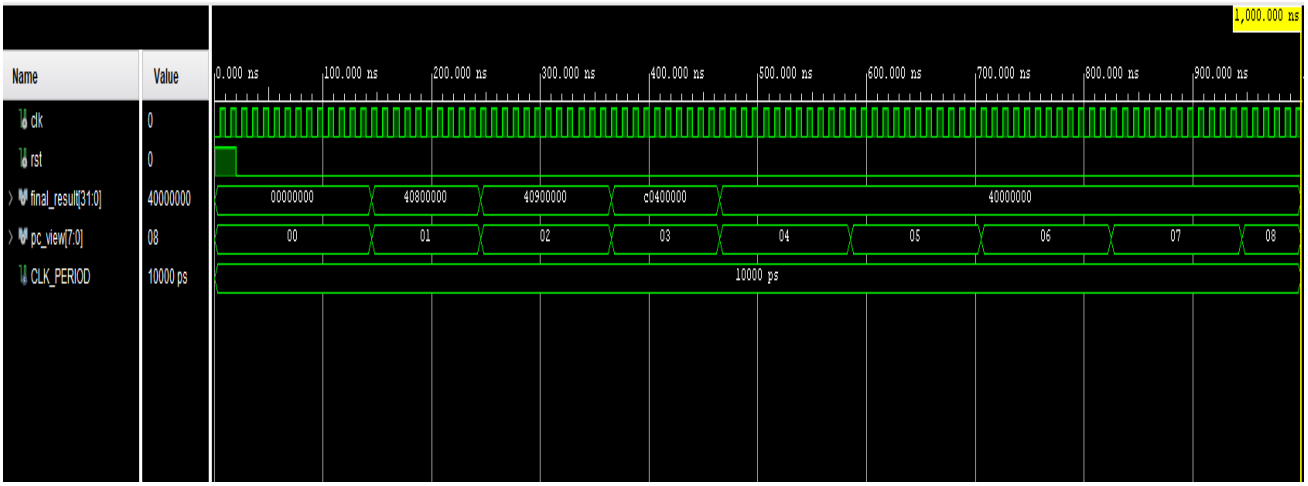
PORTS	DIRECTION	DESCRIPTION
addr_out[7:0]	out	The 8-bit address output. This value points to the current instruction in the Instruction Memory (IM)
en	in	A control signal that allows the PC to count. If en is '1', the PC increments to the next address. If en is '0', the PC holds its current value (stalls), effectively pausing the processor
clk	in	Clock
rst	in	Reset

6. TESTING AND VALIDATION

Operand 1		Operation	Operand 2		Expected Result		Obtained Result	
Base 10	Base 16		Base 10	Base 16	Base 10	Base 16	Base 10	Base 16
1.5	3FC00000	+	2.5	40200000	4.0	40800000	4.0	40800000
4.0	40800000	+	-1.5	BFC00000	2.5	40200000	2.5	40200000
-1.0	BF800000	+	3.0	40400000	2.0	40000000	2.0	40000000
-1.0	BF800000	+	-3.0	C0400000	-4.0	C0800000	-4.0	C0800000
+Inf	7F800000	+	+Inf	7F800000	+Inf	7F800000	+Inf	7F800000
-Inf	FF800000	+	-Inf	FF800000	-Inf	FF800000	-Inf	FF800000
1.0	3F800000	+	+Inf	7F800000	+Inf	7F800000	+Inf	7F800000
+Inf	7F800000	+	-Inf	FF800000	-Inf	FF800000	-Inf	FF800000
2.5	40200000	×	3.0	40400000	7.5	40F00000	7.5	40F00000
2.5	40200000	×	-3.0	C0400000	-7.5	C0F00000	-7.5	C0F00000
2.5	40200000	×	0.0	0	0.0	0	0.0	0
2.5	40200000	×	+Inf	7F800000	+Inf	7F800000	+Inf	7F800000
~2^100	71800000	×	~2^64	5F800000	+Inf	7F800000	+Inf	7F800000
~2^-100	0D800000	×	~2^-64	1F800000	0.0	0	0.0	0
1.5	3FC00000	+	2.5	40200000	4.0	40800000	4.0	40800000
1.5	3FC00000	×	3.0	40400000	4.5	40900000	4.5	40900000
-1.0	BF800000	+	-2.0	C0000000	-3.0	C0400000	-3.0	C0400000
1.0	3F800000	×	2.0	40000000	2.0	40000000	2.0	40000000







As can be observed in the simulation results, the implemented circuit meets all requirements, yielding the expected results for each of the implemented operations.

7. CONCLUSIONS

This project achieved the hardware implementation of a processor capable of executing floating-point arithmetic operations (IEEE 754), integrating complex addition and multiplication units into a synchronous and modular architecture. The system coordinates data flow through essential components such as the Control Unit (FSM), Register File, and Instruction Memory, demonstrating the correct functionality of the "Fetch-Decode-Execute" cycle. By structuring the arithmetic blocks into logical stages (alignment, normalization, rounding), the project ensures computational precision and validates the efficient interaction between the control unit and the datapath.

8. REFERENCES

[0] "Design of High Performance IEEE754 Floating Point Multiplier Using ...", IEEE Xplore, 2015. http://ieeexplore.ieee.org/document/7065627/similar

[1] "VLSI design and analysis of IEEE 754 Floating Point Multiplier", IJSET, Vol. 10 Issue 2, 2023. https://www.ijset.in/wp-content/uploads/IJSET_V10_issue2_184.pdf

[2] "754 Single Precision (32 bit) Floating Point Adder Using VHDL", IJERT, Vol. 2 Issue 7, 2013. https://www.ijert.org/research/design-of-high-performance-ieee-754-single-precision-32-bit-floating-point-adder-using-vhdl-IJERTV2IS70837.pdf

[3] "Implementation of 32 bit Floating Point Multiplier and Adder for FFT ...", IJAREEIE, Vol. 5 Issue 5, 2016. https://www.ijareeie.com/upload/2016/may/45_Implementation.pdf

[4] "Design of Floating Point Multiplier Using Vhdl", IJERD, Vol. 10 Issue 3, 2014. http://www.ijerd.com/paper/vol10-issue3/Version_1/H1037378.pdf

[5] "Area Efficient Floating-Point Adder and Multiplier with IEEE-754 ...", DIVA Portal, 2015. https://www.diva-portal.org/smash/get/diva2:789250/FULLTEXT01.pdf

[6] "DESGIN AND IMPLEMENTATION OF 32 BIT FLOATING POINT ...", IRJMETS, Vol. 4 Issue 7, 2022. https://www.irjmets.com/uploadedfiles/paper//issue_7_july_2022/28411/final/fin_irjmets1658149290.pdf

[7] "Design and implementation of floating point processor", IJARIIT, Vol. 6 Issue 3, 2020. https://www.ijariit.com/manuscripts/v6i3/V6I3-1456.pdf

[8] "754 DOUBLE-PRECISION FLOATING POINT MULTIPLIER", IJSDR, Vol. 4 Issue 11, 2019. https://ijsdr.org/papers/IJSDR1911028.pdf

[9] "Overview :: Floating Point Adder and Multiplier - OpenCores", OpenCores, 2023. https://opencores.org/projects/fpuvhdl

[10] "IEEE Standard 754 Floating Point Numbers", GeeksforGeeks, 2020. https://www.geeksforgeeks.org/computer-organization-architecture/ieee-standard-754-floating-point-numbers/

[11] "Flow-chart for floating-point IEEE-754 standard Subtraction", ResearchGate, 2024. https://www.researchgate.net/figure/Flow-chart-for-floating-point-IEEE-754-standard-Subtraction_fig1_381043777

[12] "Flow-chart for floating-point Multiplication in IEEE-754 standard", ResearchGate, 2024. https://www.researchgate.net/figure/Flow-chart-for-floating-point-Multiplication-in-IEEE-754-standard_fig2_381043777